

Attention Is All You Need

0. Abstract

우리는 RNN과 CNN이 전혀 없는 오직 attention 메커니즘만을 적용한 새로운 모델을 만들었다. 이 모델은 기존 모델의 시간과 성능을 상당한 격차로 추월시켰다.

1. Introduction

기존 모델의 recurrent 속성은 병렬화가 불가능하며, 긴 시퀀스에 취약하다. 최근 연구들은 이 제약들을 다소 해결하여 성능을 상당히 끌어올렸지만, 순차적인 계산의 제약은 여전히 남아있다. 이에 따라, 우리는 오로지 input과 output의 어텐션 메커니즘에만 기반한 모델을 만들었다.

2. Background

Transformer는 Multi-Head Attention을 통해 순차적인 계산의 long-term dependency를 해결하였다. input과 output의 관계를 이해하는 self-attention이라는 계산만을 이용한 첫 번째 번역 모델이다.

3. Model Architecture

대부분의 경쟁력 있는 번역 모델은 encoder-decoder 구조 기반으로 형성되어 있다.

encoder는 input sequence x 를 연속적인 순서 표현인 z 로 매핑한다.

이러한 z 로 디코더는 각 타임의 한 요소씩 output sequence y 를 생성한다. 각 스텝마다 모델은 자기회귀를 통해 이전 타임에 생성된 symbols를 추가 입력한다.

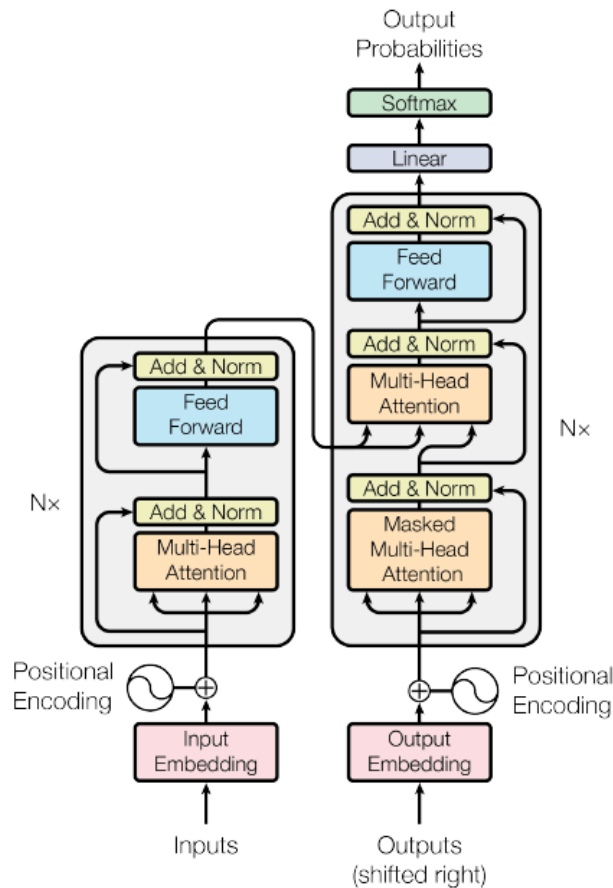


Figure 1: The Transformer - model architecture.

Transformer는 위 아키텍처와 같이 self-attention, point-wise, fully connected layers를 encoder와 decoder에 사용한다.

3.1 Encoder and Decoder Stacks

- **Encoder**

encoder는 $N(=6)$ 층으로 쌓여있다. 각 층은 두 개의 sub-layers로 구성되어 있다.

첫 번째는 multi-head self-attention 메커니즘,

두 번째는 position-wise fully connected feed-forward network이다.

우리는 두 개의 sub-layers에 residual connection을 사용하고 layer normalization을 썼다.

즉, 각 sub-layer의 output은 $LayerNorm(x + Sublayer(x))$ 이다.

이러한 방법들을 용이하게 하기 위해 d_{model} 은 512로 설정한다.

- **Decoder**

decoder 또한 $N(=6)$ 층으로 쌓여있다. decoder는 encoder의 sub-layers에 세 번째 sub-layer를 추가한다.

세 번째 sub-layer는 multi-head attention을 encoder stack의 output과 함께 사용한다.

encoder와 유사하게, 각 sub-layers에 residual connections과 layer normalization을 사용한다.

또한 self-attention sub-layer를 수정하여 다음 위치를 수반하는 것을 방지하기 위해, masking 기법을 사용했다. 이 기법은 모델이 학습할 때, 지금까지의 정보만 활용할 수 있도록 한다.

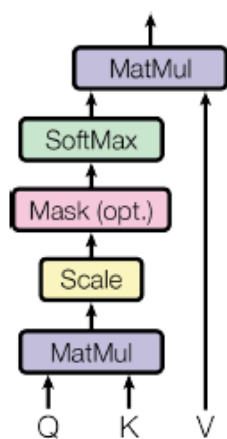
3.2 Attention

attention 함수는 query 벡터와 key-value 벡터 쌍을 매핑하여 결과를 도출하는 함수이다.

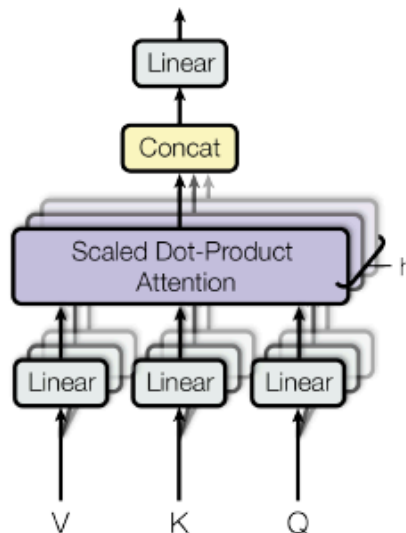
output은 해당 key를 가진 query의 compatibility function의 values들의 weighted sum으로 계산된다.

compatibility function : Query와 Key 간의 유사도 또는 관련성을 계산하여, 각 Value에 가중치를 할당하는 데 사용

Scaled Dot-Product Attention



Multi-Head Attention



왼쪽은 Scaled Dot-Product Attention

오른쪽은 병렬적인 Multi-Head Attention으로 구성된 attention layers

3.2.1 Scaled Dot-Product Attention

input은 d_k 크기의 queries와 keys, d_v 크기의 values들로 이루어져있다.

우리는 query와 모든 key들의 내적을 $\sqrt{d_k}$ 로 나눠주고, softmax 함수를 취해 values의 가중치를 얻는다.

실전에서, 우리는 query들에 attention 함수를 적용하기 위해 Q 라는 행렬로 묶어주었다. 또한 key와 value들도 이처럼 K 와 V 행렬로 묶어주었다. 우리는 output을 이렇게 계산한다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

일반적으로 attention 함수는 additive attention과 dot-product attention 이렇게 2개를 사용한다.

Additive attention은 compatibility 함수를 하나의 hidden layer와 함께 feed-forward network로 계산한다.

dot-product attention은 additive attention보다 빠르고 공간 효율적이며 행렬 곱셈에 최적화 되어있으므로 우리는 dot-product attention을 넣었다.

하지만 작은 값인 d_k 에 관해 두 방법의 성능은 유사하나, 큰 값의 d_k 에서 additive attention은 스케일링을 하지 않은 dot-product attention보다 성능이 뛰어나다.

큰 값의 d_k 는 dot-product의 크기가 매우 커져 softmax 함수를 매우 작은 기울기 값으로 유도한다. → 역전파가 잘 이루어지지 않음!

따라서 우리는 $\frac{1}{\sqrt{d_k}}$ 를 이용하여 성능을 높이하고자 하였다.

3.2.2 Multi-Head Attention

d_{model} 차원의 key, value, query를 사용하여 하나의 attention 함수를 구현하는 대신 우리는 query, key, value를 h 번의 다른 선형 변환을 통하여 d_k, d_k, d_v 차원으로 투영하는 것이 유리하다는 것을 발견했다.

각각의 query, key, value들은 attention 함수를 병렬적으로 구현하여, d_v 차원의 output 값을 산출한다.

3.2.3 Applications of Attention in our Model

인코더의 각 층은 두 가지 서브 레이어로 구성된다: 첫 번째는 Multi-Head Self-Attention 메커니즘, 두 번째는 위치별 FFN이다. 디코더도 유사한 구조지만, 인코더-디코더 어텐션 레이어가 추가되어, 디코더의 각 포지션에서 인코더의 출력 시퀀스 위치를 참고할 수 있도록 한다. 디코더의 셀프 어텐션은 마스킹을 적용하여, 각 위치가 이후 위치를 보지 못하게 막아, Auto Regressive 특성을 유지한다.

3.3 Position-wise Feed-Forward Networks

각 인코더 및 디코더 레이어는 위치별 독립적인 완전 연결 FFN을 가진다. 각 위치의 표현이 동일한 매핑을 거치지만, 서로 다른 위치끼리의 정보 공유는 없다. FFN은 두 개의 선형 변환과 중간 ReLU 활성화 함수로 구성된다. 이는 모든 포지션에 같은 파라미터를 공유해 효율적이다.

3.4 Embeddings and Softmax

입력 토큰과 출력 토큰을 각각 차원 d_{model} 인 학습 가능한 임베딩으로 변환한다. 토큰 임베딩과 출력 소프트맥스 계층은 동일한 가중치 행렬을 공유하고, 각 행은 특정 토큰의 임베딩을 나타낸다. 공유 시, 임베딩 벡터는 $\sqrt{d_{model}}$ 로 스케일링하여, 초기화 크기와 일치시킨다.

3.5 Positional Encoding

자연어 문장은 순서를 갖지만, 트랜스포머는 RNN처럼 순서를 처리하지 않으므로, 위치 정보(Positional Encoding)를 추가해 순서를 학습할 수 있도록 한다. 주기적 함수는 멀리 떨어진 토큰도 서로 다른 위치 인코딩을 가지도록 보장한다. 학습 가능한 임베딩 대신, 고정 포지셔널 인코딩을 사용한다.

4. Why Self-Attention

셀프 어텐션의 장점

- 각 단어가 모든 단어와 직접 상호작용할 수 있어 장기 의존성 문제를 완화한다
- 병렬 연산 가능해, RNN 기반 모델보다 학습 속도가 빠르다
- 각 어텐션 레이어의 연산 복잡도는 CNN보다 효율적이며,
- 셀프 어텐션은 위치 간 거리와 무관하게 상호작용하므로, 멀리 떨어진 의존성도 쉽게 학습한다

5. Training

5.1 Training Data and Batching

- 학습 데이터: WMT 2014 영어-독일어 번역 데이터셋 (450만 문장 쌍)
- 추가 실험: WMT 2014 영어-프랑스어 번역 데이터셋 (3600만 문장 쌍)
- 각 배치는 유사한 길이의 문장끼리 그룹화해 패딩 낭비를 최소화하고 학습 속도를 높임

5.2 Hardware and Schedule

- 8개의 NVIDIA P100 GPU에서 학습

- 배치 크기: 각 GPU당 약 25,000 토큰 처리
- 12시간 이내에 영어-독일어 모델을 학습 완료

5.3 Optimizer

- 옵티마이저: **Adam** 사용
- 초기 학습률: 10-4
- 워밍업 스텝: 4000 스텝
- 초기에 큰 학습률로 인해 불안정해지는 문제를 방지하고, 이후 점진적 감소로 안정성 확보

5.4 Regularization

- 드롭아웃: 어텐션 가중치, 피드포워드 레이어, 서브레이어 출력을 대상으로 0.1 확률로 드롭아웃
- 라벨 스무딩(label smoothing) 적용: 정답 토큰 확률을 0.9로, 나머지를 0.1로 분산 - 단순 원핫보다 일반화 성능 향상.

6. Conclusion

- 완전한 어텐션 기반 모델인 Transformer를 제안
- 순차 처리 없이 병렬 연산이 가능해 학습 및 추론 속도가 대폭 향상
- 기존 Seq2Seq 모델보다 성능이 우수하며, 특히 긴 문장 처리 능력이 뛰어남
- 기계번역 외에도 다양한 NLP 태스크로 확장 가능성을 제시
- 이후 자연어처리 모델 연구에 기본 아키텍처로 자리 잡는 계기가 된 논문